

Proyecto Programado II – Backend distribuido en Microservicios

Entrega: 08 Junio 2026 (Semana 16)

Hora de entrega: 07:00AM

Curso: Diseño de Software

Profesor: Marcos Rodríguez

Valor: 35%

Objetivo

El objetivo de este proyecto es que los estudiantes desarrollen competencias en la creación de aplicaciones empresariales distribuidas basadas en microservicios y en la implementación de servicios en la nube.

Los grupos de trabajo serán los mismos asignados en la Fase #1.

Alcance del Proyecto

1. Microservicios

- Se deberán desarrollar mínimo tres microservicios independientes, desplegados en Azure App Services, que expongan los endpoints definidos en el proyecto anterior y sean consumidos por la aplicación frontend y/o móvil.

- *Requisitos tecnológicos:*

* Uno de los microservicios debe ser implementado en Spring Boot, siguiendo la arquitectura hexagonal vista en clase.

* Los otros microservicios deben utilizar tecnologías distintas entre sí, a libre elección del equipo (ej. Node.js con Express, Python con FastAPI, .NET Core, etc.).

* Un microservicio específico deberá encargarse de la gestión de notificaciones (envío, consulta y persistencia).

2. Seguridad

- Exposición centralizada: todos los microservicios deberán ser publicados exclusivamente a través de Azure API Management (APIM).

- Comunicación segura: la conexión entre APIM y cada microservicio deberá estar protegida mediante certificados digitales, de forma que solo APIM pueda invocar los servicios backend.

- Se deberá documentar la configuración de APIM, mediante una colección de Postman completamente funcional:

* Políticas de seguridad aplicadas.

* Autenticación y autorización.

* Manejo de claves o tokens.

* La colección de Postman deberá ser completamente ejecutable y contener al menos una prueba para cada endpoint.

3. Notificaciones

- Se debe implementar un servicio de notificaciones asíncronas asociado a una transacción elegida por el equipo (ejemplo: confirmación de compra, alta de usuario, actualización de estado).

- Tipos de notificación requeridos:

1. Visualización en tiempo real en la aplicación web/móvil (ej. *WebSockets* o *SignalR*).

2. Notificaciones push en la aplicación móvil.

- Flujo esperado:

1. El microservicio que origina el evento enviará un mensaje a una cola en Azure.

2. El microservicio de notificaciones procesará el mensaje y ejecutará las siguientes acciones:

a. Insertar la notificación en la base de datos (persistencia para consulta posterior).

b. Emitir la notificación en tiempo real hacia la aplicación web.

c. Enviar la notificación push a la aplicación móvil.

4. Servicios de almacenamiento

- Cada microservicio deberá integrar un servicio de almacenamiento diferente, de modo que en el proyecto se utilicen al menos tres tipos de servicios de datos en Azure.

- Ejemplos válidos (a elección del equipo):

* Relacional (SQL): Azure SQL Database, PostgreSQL, MySQL, otra.

* NoSQL: Azure Cosmos DB (Mongo, Cassandra, Table API).

* Mensajería/colas: Azure Queue Storage o Azure Service Bus.

- Se espera que los estudiantes justifiquen la elección de cada servicio, en función de los requerimientos del microservicio.

Entregables

a. Código fuente de los tres microservicios, con documentación cada uno en GitHub + Actions.

b. Configuración de API Management con documentación de seguridad aplicada (Postman + Swagger).

c. Demostración funcional del flujo de notificaciones en tiempo real y notificaciones push móviles.

d. Evidencia del uso de tres servicios de almacenamiento distintos en Azure.

e. Colección de Postman exportada con todos los endpoints funcionales en formato JSON.

5. Documentación (Mediante un video demostrativo, no más de 5min), debe:

- Asegurar un formato visual en todo momento, explicativo, claro, con imágenes y explicación en todo momento.

- Seguir un orden lógico (un guión) al presentar las partes de la solución

- Presentar:

- Arquitectura general del sistema.

- Demostración de flujos de negocio principales de la aplicación según el usuario.

Nota: Solo se asignará nota a los proyecto bajo revisión tipo defensa donde asistan todos los miembros del equipo.

Criterio	0	1	2	3	4
Microservicios implementados según especificación	No hay microservicios implementados	Microservicios incompletos o con errores graves	Microservicios funcionales con errores menores	Microservicios completamente funcionales y bien estructurados	Microservicios funcionales, óptimos y escalables
Uso correcto de Spring Boot y arquitectura hexagonal	Spring Boot no utilizado o arquitectura incorrecta	Spring Boot usado parcialmente, arquitectura con fallas	Spring Boot correcto, arquitectura parcialmente aplicada	Spring Boot y arquitectura hexagonal bien aplicados	Spring Boot y arquitectura hexagonal aplicados de forma excelente y escalable
Uso de tecnologías diversas para microservicios adicionales	No se utilizan tecnologías diversas o incorrectas	Solo se usa una tecnología adicional	Tecnologías diversas aplicadas parcialmente	Tecnologías diversas correctamente implementadas	Tecnologías diversas aplicadas de forma óptima y escalable
Microservicio de notificaciones funcionando correctamente	Microservicio de notificaciones no funciona	Microservicio de notificaciones parcialmente funcional	Microservicio de notificaciones funcional con errores menores	Microservicio de notificaciones completamente funcional	Microservicio de notificaciones eficiente, escalable y robusto
Seguridad y exposición vía Azure APIM con certificados	No se configuró seguridad o APIM	Seguridad básica implementada con errores	Seguridad APIM correcta, certificados parcialmente configurados	Seguridad APIM bien configurada y certificados aplicados correctamente	Seguridad APIM aplicada de manera óptima con mejores prácticas

Implementación de notificaciones asíncronas (cola y flujo)	No hay notificaciones asíncronas implementadas	Notificaciones asíncronas muy limitadas	Notificaciones asíncronas funcionando pero con errores menores	Notificaciones asíncronas funcionando correctamente	Notificaciones asíncronas eficientes, escalables y confiables
Persistencia de notificaciones en base de datos	No se persisten notificaciones	Persistencia limitada o con errores	Persistencia de notificaciones funcional con detalles menores	Persistencia de notificaciones correcta	Persistencia de notificaciones robusta y optimizada
Envío de notificaciones web y push móvil	No se envían notificaciones	Notificaciones solo parciales	Envío web y push funcional con errores menores	Envío web y push funcionando correctamente	Envío web y push eficiente y escalable
Uso de tres tipos de servicios de almacenamiento diferentes	No se usan tres tipos de almacenamiento distintos	Solo se usan dos tipos de almacenamiento	Tres tipos de almacenamiento usados, con configuraciones parciales	Tres tipos de almacenamiento correctamente implementados	Tres tipos de almacenamiento aplicados con buenas prácticas y optimización
Documentación clara y completa del proyecto	Documentación inexistente o incompleta	Documentación incompleta o poco clara	Documentación clara, detalles faltantes	Documentación completa y clara	Documentación excelente, detallada, con diagramas y buenas prácticas